

AS DE FRONTEND

Nome do Aluno: Matheus de Oliveira dos Santos

Disciplina: Desenvolvimento Front-End

1. Estrutura do Projeto

Organização dos Arquivos

O projeto foi organizado de forma direta na raiz do diretório para facilitar a execução local instantânea, mantendo os arquivos de código integrados e uma pasta dedicada para o relatório técnico

```
projeto
├── index.html
├── js
│   └── script.js
└── docs
    └── documentacao.pdf
```

Função de Cada Arquivo

index.html

Contém a estrutura estática global da aplicação. Ele carrega os campos de entrada, as colunas organizadoras de tarefas, os contadores automatizados e realiza a importação do framework utilitário Tailwind CSS através de sua CDN estável

script.js

Centraliza toda a inteligência e o comportamento assíncrono do sistema. Ele faz o controle dos eventos de clique, executa a verificação para impedir campos em branco, manipula dinamicamente os contadores e renderiza os novos elementos no fluxo visual da tela.

docs

Relatório técnico contendo a análise do código e a explicação dos conceitos de programação utilizados.

Estrutura Geral da Aplicação

A interface adota a estrutura conceitual de um painel de controle único (*dashboard*). No topo, posiciona-se um cabeçalho descritivo com a listagem de legendas cromáticas das prioridades (Alta como vermelho, Média como amarelo e Baixa como verde). Logo abaixo, está o formulário de cadastro estruturado de forma horizontal. A base da aplicação é regida por um sistema de grade que divide o layout em duas áreas paralelas idênticas: a lista para tarefas em estado pendente e a área destinada ao registro de histórico concluído.

2. Explicação dos Métodos e Conceitos Utilizados

Seleção de Elementos

```
document.getElementById()
```

Explicação: É um comando usado para buscar uma tag do HTML dentro do arquivo JavaScript através do seu **id** exclusivo. Serve para dar ao script o controle sobre um elemento específico da página.

Exemplo no projeto: Usado no início do arquivo para selecionar os campos de entrada e os contadores:

```
const inputTarefa = document.getElementById('inputTarefa');
```

```
document.querySelector()
```

Explicação: Funciona como um buscador que localiza elementos usando as mesmas regras de seleção do CSS (como buscar por nomes de classes usando o ponto). É muito útil para encontrar elementos que estão dentro de outros.

Exemplo no projeto: Usado para achar os botões de controle dentro de cada card de tarefa criado:

```
const botaoExcluir = card.querySelector('.botao-excluir');
```

Criação e Manipulação de Elementos

```
document.createElement()
```

Explicação: Serve para criar uma nova tag HTML do zero usando o JavaScript. Esse elemento fica guardado na memória e só aparece na tela do site quando é inserido em algum lugar da página.

Exemplo no projeto: Usado para criar o bloco quadrado que serve de base para cada tarefa:

```
const card = document.createElement('div');
```

```
element.appendChild()
```

Explicação: Coloca um elemento novo dentro de outra tag que já existe na página, adicionando-o no final da lista de elementos daquele local.

Exemplo no projeto: Usado para inserir o card da nova tarefa dentro da lista de tarefas pendentes:

```
listaPendentes.appendChild(card);
```

```
element.classList.add()
```

Explicação: Adiciona uma nova classe de estilo a um elemento. No projeto, foi usado para ocultar avisos da tela aplicando a classe utilitária do Tailwind.

Exemplo no projeto: Usado para esconder a mensagem de erro quando o usuário digita algo válido:

```
mensagemErro.classList.add('hidden');
```

```
element.classList.remove()
```

Explicação: Remove uma classe de estilo que já estava aplicada a um elemento.

Exemplo no projeto: Usado para fazer a mensagem de erro oculta reaparecer na tela quando o usuário tenta cadastrar uma tarefa vazia:

```
mensagemErro.classList.remove('hidden');
```

Manipulação de Eventos

```
element.onclick
```

Explicação: É um recurso que faz o site monitorar quando o usuário clica com o mouse em cima de um botão específico. Quando o clique acontece, o JavaScript executa a função que foi programada para aquele botão.

Exemplo no projeto: Usado para fazer o botão de excluir apagar a tarefa correspondente:

```
botaoExcluir.onclick = function() { card.remove(); };
```

Manipulação de Estilos

Aplicação dinâmica de classes do Tailwind CSS e `element.style`

Explicação: Consiste em alterar o visual dos elementos em tempo real enquanto o usuário mexe no site. No projeto, em vez de alterar estilo por estilo manualmente com `.style`, usamos a propriedade `.className` para injetar pacotes de classes do Tailwind CSS de uma só vez, aplicando as bordas coloridas nos cartões de acordo com a prioridade selecionada.

Exemplo no projeto:

```
corDoCard = "border border-green-500 bg-neutral-950";  
card.className = "p-3 rounded-lg shadow-sm flex justify-between  
items-center " + corDoCard;
```

Remoção e Movimentação de Elementos

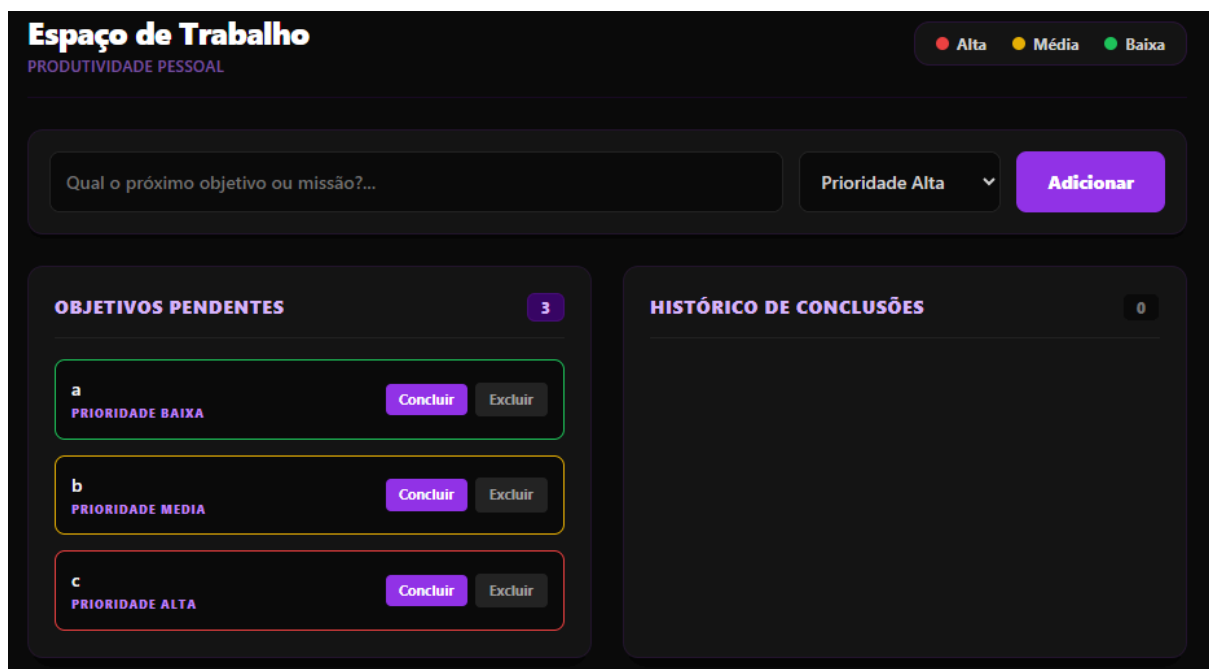
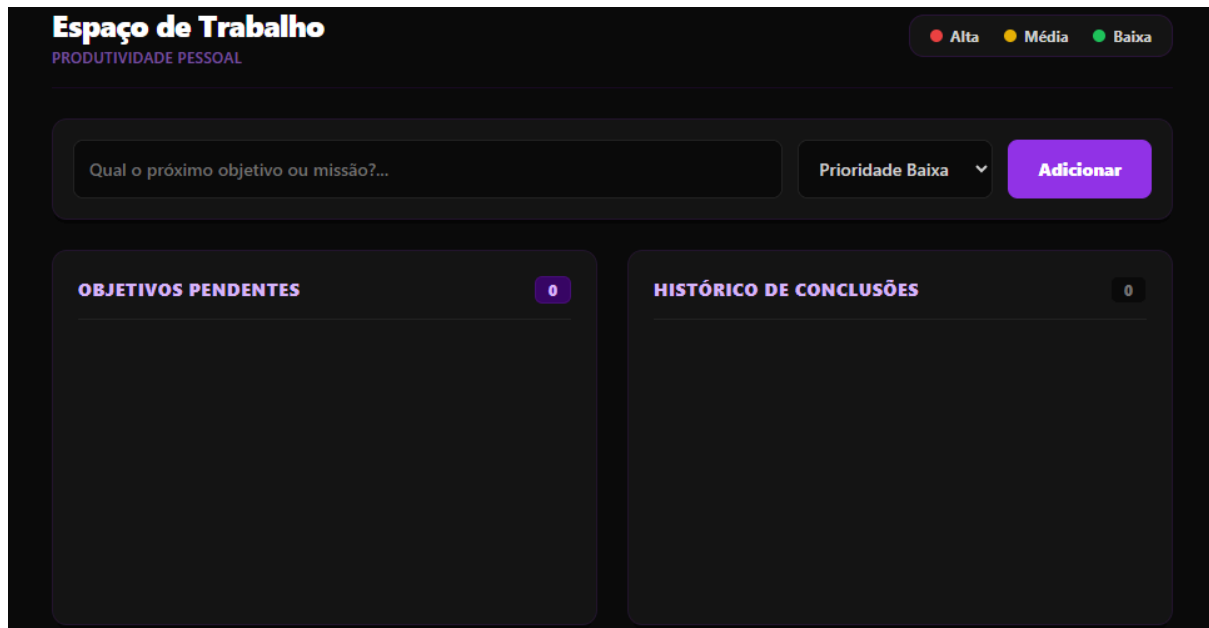
element.remove() e Transferência entre Listas

Explicação: O comando **remove()** serve para apagar um elemento do site de forma definitiva. Para fazer a movimentação de uma tarefa de pendente para concluída, o card original é excluído da primeira coluna com o **.remove()** e a função cria um card totalmente novo com a data e hora na coluna de histórico.

Exemplo no projeto:

```
card.remove();moverParaHistorico(textoTarefa,prioridadeSelecionada);
```

3. Capturas de Tela



Qual o próximo objetivo ou missão?...

Prioridade Alta

Adicionar

OBJETIVOS PENDENTES

0

HISTÓRICO DE CONCLUSÕES

3

c

Concluído em 01/07/2026 às 16:14

ALTA

b

Concluído em 01/07/2026 às 16:14

MEDIA

a

Concluído em 01/07/2026 às 16:14

BAIXA

Espaço de Trabalho

PRODUTIVIDADE PESSOAL

Alta
Média
Baixa

Qual o próximo objetivo ou missão?...

Prioridade Alta



Adicionar

OBJETIVOS PENDENTES

0

HISTÓRICO DE CONCLUSÕES

3

c

ALTA

Concluído em 01/07/2026 às 16:14

b

MEDIA

Concluído em 01/07/2026 às 16:14

a

BAIXA

Concluído em 01/07/2026 às 16:14

4. Considerações Finais

O que foi aprendido

O desenvolvimento deste projeto ajudou a entender na prática como o JavaScript se conecta com o HTML para criar páginas interativas. Foi possível aprender como capturar as ações do usuário, validar se as informações digitadas estão corretas e criar elementos visuais na tela apenas usando programação, sem precisar deixar tudo fixo no código do HTML. O uso do Tailwind CSS também foi importante para compreender a aplicação prática de layouts responsivos.

Principais dificuldades encontradas

O principal desafio no desenvolvimento foi ajustar a lógica dos contadores para que eles fizessem as contas matemáticas certas de adição e subtração no momento exato em que uma tarefa era criada, movida para o histórico ou deletada do site.

Como os problemas foram resolvidos

Os problemas dos botões e dos contadores foram resolvidos colocando as funções de clique e as contas de somar ou subtrair diretamente dentro da função principal de criação das tarefas. Dessa forma, cada vez que o site cria um card novo, ele já nasce sabendo o que fazer quando for clicado e atualiza os números na tela instantaneamente. O visual das prioridades foi ajustado aplicando classes de borda completa ao redor das caixas para destacar bem a cor verde na prioridade baixa.